

☒ CORRECTIONS v2 — BioText Transformer

Résumé des bugs corrigés et des améliorations apportées à `biotext_light.py`, `start.py` et `hardware_check.py`. L'original est conservé dans `biotext_light_ORIGINAL_backup.py`.

Résultats avant / après (CPU, 2 threads)

Critère	v1 (original)	v2 (corrigé)
Loss initial	40.7 (aberrant)	5.547 (= log 256)
Loss final (100 étapes)	16.0	2.51
Loss final (300 étapes)	—	0.675
Arêtes après 1er sommeil	[0, 0] puis [1, 1]	[157, 159] → oscille 138–163
Loss replay pendant sommeil	non mesuré	baisse à chaque sommeil (11/11)
Génération	Le cerveauuuuuuuu]]]]]]]	Le cerveau contienux ... synapsique ... L'ègines hebbienes et ...
Sauvegarde	crash (/mnt/user-data/outputs inexistant)	./biotext_light_weights.pt (0.61 Mo)
Temps / étape	~90 ms	65 ms (batch 2× plus grand)

Bugs corrigés

1. Toutes les arêtes élaguées après le sommeil

Cause : score d'importance absolu ($\text{gate} \times |\text{mean}(\text{src}) \cdot \text{mean}(\text{dst})| \approx 0.05\text{--}0.09$) comparé au seuil `tau_prune = 0.15` → 100 % élaguées, 4 recréées, re-élaguées.

Correctifs :

- `Score = gate × |corrélacion de Pearson(src, dst)|` sur le batch ($\in [0, 1]$), lissé par EMA.
- **Normalisation par le max** des arêtes actives avant comparaison à `tau_prune`.
- **Garde-fous :** max 25 % élaguées par sommeil (`prune_frac_max`), plancher `min_edges = 32`, seules les arêtes « mûres » (≥ 5 mises à jour) sont candidates.
- **Croissance homéostatique :** `grow_step + n_pruned // 2` nouvelles arêtes → le nombre d'arêtes peut monter ou descendre, sans jamais s'effondrer.

- Nouvelles arêtes initialisées avec l'importance médiane (pas 0) → pas d'élagage immédiat.
- **Tenseurs à taille fixe** `max_edges` + **masque** active : les `nn.Parameter` ne sont plus recréés → l'optimiseur AdamW suit toujours les bons paramètres (en v1, les arêtes recréées n'étaient jamais entraînées). État Adam remis à zéro pour les slots recyclés.
- Les logits de gates ont leur propre groupe d'optimisation ($lr \times 5$, sans weight decay).

2. Loss initial aberrant (≈ 40 au lieu de 5.5)

Cause réelle : `nn.Embedding` initialisé en $N(0, 1)$ et lié à la tête de sortie

→ logits d'écart-type ≈ 8.5 . Le Hebbian non contrôlé aggravait ensuite la dérive.

Correctifs : embeddings et positions en $N(0, 0.02^2)$ (style GPT) → loss initial 5.547.

3. Hebbian/STDP non normalisé (explosion des poids)

Correctifs (`FastHebbianSTDP`) :

- Activations standardisées (z-score par feature) → Hebbian = matrice de corrélation bornée.
- STDP avec **traces temporelles causales le long de la séquence** (noyau $\exp(-\Delta t/\tau)$) :
pré avant post → LTP, post avant pré → LTD (la v1 utilisait des moyennes sans notion de temps).
- La contribution hebbienne est stockée séparément : `W_eff = W_gradient + H`.
H décroît (`decay_w`, oubli synaptique) et sa norme est **plafonnée à 5 % de $\|W\|$** .
Les poids appris par gradient ne sont donc ni érodés ni explosés.
- Appliqué à Q, K, V et aux deux matrices de la FFN (la v1 : Q et K seulement).

4. Génération dégradée

Causes : (a) **absence de masque causal** — le modèle voyait le token futur qu'il

devait prédire, donc le loss baissait artificiellement et la génération était du bruit ;

(b) **argmax pur** → répétitions uuuuu.

Correctifs : `F.scaled_dot_product_attention(..., is_causal=True)` ; génération avec pénalité de répétition + **sampling top-k** / température (et mode greedy) ; encodage UTF-8 du prompt et du corpus (accents corrects).

5. Chemin de sauvegarde inexistant

`./biotext_light_weights.pt` (relatif au projet) via `model.save()` /

`FastBioTextTransformer.load()`.

Le checkpoint contient aussi la composante hebbienne H et la config.

`hardware_check.py` écrit `config_example.py` dans le dossier du projet et lit le CSV composants en relatif (`--csv chemin.csv` optionnel).

6. API PyTorch dépréciées

`torch.amp.autocast("cuda", dtype=torch.float16)` et `torch.amp.GradScaler("cuda")`.
`torch.load(..., weights_only=False)` explicite.

7. Sparse coding trop agressif (bug non listé)

3 % de 64 = 1 seule feature conservée dans l'attention. Ajout d'un plancher
`topk_min = 4` (6.2 % attention, 3.1 % FFN — mesurés et affichés).

8. Divers

- Boucle python sur les arêtes remplacée par `index_add` vectorisé (GPU-friendly).
- `grad_accum` réellement implémenté (batch effectif 16).
- Loss affiché = moyenne sur 10 étapes (moins bruité) ; loss replay avant/après sommeil.
- `torch.set_num_threads(os.cpu_count())` en mode CPU ; TF32 + `torch.backends.cudnn.benchmark` sur GPU.
- Flags d'ablation `use_hebbian`, `use_sleep`, `use_sparse` (CLI : `--no-hebbian`, etc.).
- `start.py` : chemins relatifs à `__file__`, overrides appliqués avant construction, `--resume`, `--prompt`, `--cpu`, ajustement automatique de `n_heads`.
- `hardware_check.py` : plus de dépendance `wmi`, mini-benchmark `matmul`, recos RTX 3060 à jour.

Optimisations ciblées RTX 3060 + Core Ultra 5 245KF

- AMP fp16 + GradScaler, TF32 activé, attention fusionnée (Flash/SDPA).
- Modèle par défaut (88k params) : quelques secondes pour 300 étapes sur GPU.
Profil recommandé : `python start.py --d-model 128 --n-layers 4 --n-heads 4 --d-ff 256 --batch-size 32 --max-edges 512 --steps 1000`.
- Mode CPU : tous les cœurs utilisés, ~65 ms/étape sur 2 threads Xeon → nettement moins sur le 245KF.

Étude d'ablation rapide (300 étapes, CPU, seed 42)

Variante	Loss final
Complet (défaut)	0.675
Sans Hebbian/STDP	0.763
Connectivité figée (32 arêtes)	0.848
<code>topk_min = 2</code>	0.946